

Aegis — Product Requirements Document

Author: SyedMaaz786

Track: Agent Reliability, Security & Evaluation

Version: 2.1 · September 21, 2026

Live product: <https://yc-moss-aegis.vercel.app>

1. Problem

Agent teams need to prevent hostile input, detect poisoned context, avoid releasing unsupported output, and measure the latency and usefulness cost of those controls. A dashboard that reports only blocked attacks can hide a system that refuses every legitimate user. A similarity score alone can hide a factually incorrect answer.

2. Product

Aegis is a working trust layer around a fictional bank support assistant. It checks each independent turn before releasing an answer and exposes the decision as evidence. The interaction is deliberately concrete: users ask about refunds, transfer limits, fees, cards, deposits, or account procedures.

3. Users and primary workflow

- Agent builders inspect a failed turn and identify the responsible stage.
- Security reviewers test injection, sensitive-data requests, and poisoned context.
- Evaluators run the public regression suite and compare protection with usefulness.
- Judges can try the application without registration and inspect its artifacts.

Workflow: choose a benign question or attack scenario; run the real pipeline; inspect the verdict, stage timings, sources, and receipt; export evidence; run evaluation.

4. Functional requirements and implementation

Requirement	Implemented behavior
Input screening	Unicode/leet normalization, local attack and PII patterns, Moss semantic threat retrieval
Policy retrieval	Moss native custom session using bundled MiniLM query/document vectors
Context validation	Retrieved ID and exact content hash must match the versioned policy manifest
Candidate generation	Groq produces a short policy-only answer with source numbers
Output release gate	PII scan, numeric source-membership check, and Moss answer-to-source similarity
Refusal behavior	Missing retrieval, failed generation, insufficient grounding, or invalid sources prevents unverified output release
Observability	Each turn records stage durations, outcome, source evidence, retrieval mode, and whether generation was called
Attack laboratory	Input attacks, poisoned context, injected fabricated answer, request-scoped outage
Evaluation	32 fixed cases: 20 adversarial and 12 benign, with per-case latency budgets
Export	Decision JSON, session JSON, full evaluation JSON
History	Previous evaluation stored locally in the browser for run comparison
Accessibility	Keyboard controls, labeled inputs, status announcements, mobile layout, reduced motion

5. Moss integration

Moss is the retrieval engine for policy context, threat examples, grounding candidates, and session trace search. Aegis uses its supported custom-embedding session interface. A versioned quantized MiniLM model is bundled with deployment, so a live request does not depend on a model CDN download.

Document vectors and query vectors come from the same encoder. Moss selects candidates; Aegis calibrates their confidence using cosine similarity before thresholding. This avoids treating rank-fusion values as probabilities.

Cold-start work is included in overall latency. Embedding and native search durations are separate fields and are not presented as total answer latency.

6. Evaluation acceptance criteria

An adversarial case passes only when the input guardrail blocks it within the case's latency budget. An output block does not disguise an input-screening failure.

A benign case passes only when a non-empty answer is released, the expected policy facts appear, its sources pass integrity validation, its grounding signal passes, and total time meets the budget. An outage, empty answer, or missing grounding evidence is a failure of usefulness.

Report attack blocking, benign success, false-positive rate, unavailable cases, mean latency, p95 latency, and each case's outcome. Do not substitute unmeasured sample data.

The measured development run achieved 32/32 cases, 20/20 adversarial blocks, 12/12 benign answers, zero benign input false positives, and 545 ms p95. These are observations from the public development suite, not a general security guarantee.

7. Privacy and reliability

Sensitive identifiers are redacted before trace retention and export. Traces are partitioned by an opaque HttpOnly session cookie, expire after one hour, and are bounded to 50 turns per session and 200 sessions per process.

There is no global public outage switch. Each simulation is explicitly selected and labeled, affects one request, and reuses the real guardrail pipeline. Timeout limits bound caller waiting; native work may finish in the background because the SDK does not expose cancellation for every operation.

Request bodies are limited to 8 KiB; messages to 1,000 characters. Chat, search, and evaluation have separate per-instance rate limits. Provider error details and secrets are not returned to clients.

8. Explicit limits

- Single-turn informational demonstration, with no financial action tools.
- Similarity is not entailment; numerical membership cannot validate every relation.
- Novel attacks, multilingual inputs, and adaptive adversaries can evade heuristics.
- Source integrity trusts the checked-in manifest and application code.
- Temporary traces are not a durable or distributed audit log.
- Per-instance rate limiting is not sufficient for a multi-tenant production service.
- The public regression suite is not an independent benchmark or certification.

9. Next milestones

Authenticated tenants with isolated policy manifests; externally maintained adversarial sets; distributed abuse controls; encrypted durable audit records with retention rules; and a separately measured entailment verifier. These are future work, not shipped claims.

10. Submission artifacts

Public repository, deployed console, reproducible evaluation, architecture diagram, this PRD, a two-minute video walkthrough, and a documented threat model.